



Research + Build + Present

~2 days

30-40 min + Q&A

Deep Research Agent System

Research Engineer Take-Home Assessment

1. Overview

Build a system that conducts **deep research** on a given topic and produces a structured, evidence-backed research report. The system should iteratively investigate, handle conflicting sources, and synthesize across multiple research threads.

How you architect this is up to you. Multi-agent, single-agent with tools, a hybrid—we care about the reasoning behind your approach, not whether it matches a particular pattern. If you think the obvious approach is wrong, say so and show us why.

⚠ This is our only interview round. We value **depth of thought over breadth of features**. We will probe your understanding of every design choice. If you can't explain why a piece of your system exists, it shouldn't be there.

ℹ AI tooling is allowed, including open-source agent frameworks (LangGraph, CrewAI, AutoGen, etc.). Use whatever helps—but be ready to explain what it buys you and where it gets in the way.

2. Requirements

We expect two things: a **full design** and an **MVP implementation**. Design the system you'd actually want to build, then implement just enough to demonstrate your core ideas in ~2 days.

2.1 Full Design

Present a complete architecture for a production-grade deep research agent system. Your design should reflect the depth you chose in [Choose Your Depth](#)—we want to see deep,

production-grade thinking in your area of expertise, not a surface-level survey of every component.

To give you a sense of scope, here are a few dimensions we'd expect a strong design to grapple with—but these are examples, not a checklist:

Example design dimensions

Architecture & orchestration—How is work structured and coordinated? What talks to what, and why?

Memory & context—How does the system manage state, context limits, and decide what to keep vs. discard?

Tool design—What tools does the system have, and what happens when they fail?

Quality & reliability—How does the system stay honest when sources conflict or information is missing?

What's **not** on this list matters as much as what is. A production system has concerns we haven't mentioned—identifying and addressing the ones that matter most for **this** problem is part of the exercise.

- ✔ If you want to ask us a clarifying question about a design choice—that's the point. Make a decision instead.

2.2 MVP Implementation

Build a runnable MVP that demonstrates your core idea end-to-end. You decide what's in the MVP and what stays in the design—that prioritization itself is something we evaluate.

Minimum bar

Demonstrates your **core architectural idea end-to-end**—not a toy, but enough to prove the concept works

Uses at least one **real external data source** (e.g., web search, document retrieval, API)

Final output must **separate claims from evidence** and indicate confidence and source agreement

Include **evidence that your system works**—example outputs with your own quality assessment, failure cases you found, and how you addressed them

Include at least one **comparison or ablation** (e.g., baseline vs. your approach, with vs. without a key component) that isolates the impact of a design choice

Beyond this, the shape of your MVP is yours to define. If your most interesting idea is a training pipeline, a novel evaluation framework, or a single agent that outperforms a naive multi-agent setup—build that. We'd rather see one well-executed idea than a feature checklist.

 A clean, well-reasoned MVP beats an ambitious but broken prototype.

 We're interested in your process, not just your output. Consider keeping a brief log of key decisions, tools you used, and how your thinking evolved. What did you try that didn't work? What changed your mind? This tells us more than polished results alone.

2.3 Presentation (30-40 min + Q&A)

Structure the presentation however you think best communicates your work. We want to understand your full design (not just what you built), the reasoning behind your choices, and how your work connects to the broader research landscape. Include a demo—live or recorded.

We will ask hard questions. Come prepared to defend your design, acknowledge its weaknesses, and explain what you'd do differently with more time.

2.4 Choose Your Depth

This project can go in very different directions depending on your expertise. We want to see you go deep on what you're best at—not spread thin across everything. Pick the angle that lets you do your most impressive work.

 We encourage you to survey the current literature on deep research agents and multi-agent systems. Our daily work is applied research, and we're looking for someone who does the same.

QUESTIONS WORTH THINKING ABOUT—REGARDLESS OF TRACK

When should a plan be a commitment vs. a suggestion?

What should agents share, and what should they not?

Dead ends, contradictions, hallucinations—how does the system stay honest?

Not all work is equally hard. What does that imply?

Vague question in, concrete research out. What happens in between?

When does adding another agent help vs. hurt?

How do you prevent agents from converging on a comfortable, wrong answer?

When do you stop searching?

RESEARCH & POST-TRAINING

If your strength is model building, lean into it. There's a rich and fast-moving research landscape here—we want to see that you know it and can apply it. A few starting points to give you the flavor:

RL for interleaved search and reasoning (e.g., Search-R1—what would you change for a multi-agent setting?)

Process reward models for research quality—step-level signals vs. outcome-only, and what makes a good intermediate reward for a research agent

Multi-agent self-play for reasoning (e.g., MARS)—can agents improve each other through debate or cross-verification?

Training self-correction into agents (e.g., SCoRe)—how do you teach a model to catch its own errors mid-research?

Test-time compute scaling—spending more inference on harder sub-questions. When is this worth it?

Training with noisy retrieval (e.g., Finetune-RAG)—real sources are messy,

ENGINEERING & INFRASTRUCTURE

If your strength is systems and infra, show us what you can build. Example directions:

Agent orchestration at scale—scheduling, resource management, fault tolerance

Observability and debugging systems for multi-agent workflows

Tool integration frameworks and dynamic tool selection

Prompt tuning infrastructure—versioning, A/B testing, and systematic evaluation of prompt strategies across agents

Latency optimization and cost-aware execution strategies

Deployment pipelines and production-grade agent infrastructure

QUESTIONS FOR THIS TRACK

How do you debug a multi-agent workflow when something goes wrong three steps ago?

What does graceful degradation look like when an agent or tool fails mid-research?

What would it take for the system to get better over time without retraining?

conflicting, incomplete. How do you train for that?

QUESTIONS FOR THIS TRACK

Where does prompting hit its ceiling and training become necessary?

If you could train or fine-tune a model specifically for this task, what would you change about it?

How would you generate training data for a research agent when there's no labeled dataset?

No ground truth exists for a research report. How do you know yours is good?

*These are examples, not a checklist. We're more interested in directions we haven't listed—what would **you** try?*

These aren't mutually exclusive—blend them if that's where your best work lives. The point is to go deep somewhere rather than shallow everywhere.

- ✔ We're looking for a **wow moment**—the idea, design, or result that makes us say "this person really knows their stuff." Show us the thing only someone with your specific expertise could produce.

3. Test Cases

Here are four example prompts of varying difficulty. Use them, modify them, or replace them entirely—what matters is that you demonstrate your system on prompts that reveal something interesting about how it works, including where it struggles.

EXAMPLE 1 **Straightforward**

"What are the real-world risks and benefits of using synthetic data to train or fine-tune large language models? Focus on data quality, bias, and evaluation."

EXAMPLE 2 Contradictory sources

"Is chain-of-thought prompting an effective reasoning strategy for LLMs, or does it primarily improve output formatting? The literature disagrees—find the real fault lines and explain what accounts for the conflicting results."

EXAMPLE 3 Sparse & emerging

"What is the current state of inference-time compute scaling for LLM reasoning? Separate what has been empirically validated from what is still speculative, and identify where the evidence is too thin to draw conclusions."

EXAMPLE 4 Multi-modal output

"Map the research landscape of multi-agent LLM systems. Produce a structured report that includes a visual taxonomy or design graph showing the major architectural patterns, how they relate, and where the open problems are."



Your choice of test cases tells us how well you understand your own system. What prompts would break it, expose its weaknesses, or reveal interesting behavior?

4. Evaluation

Here are some of the dimensions we evaluate. This is not an exhaustive list.

Architectural Judgment

Can you justify your agent boundaries and orchestration?

Depth of Understanding

Explain every component: why it exists, how it fails, what you'd change.

Technical Fundamentals

Working knowledge of ML concepts (embeddings, retrieval, fine-tuning, evaluation)

Engineering Quality

We review the repo before the interview. Make it count.

metrics) and engineering basics (clean code, error handling, reproducibility).

Research Depth

Do you know the landscape, and does it show in your output? We want grounded choices and genuine multi-source synthesis, not concatenated summaries.

Experimental Rigor

Can you design controlled comparisons, ablations, or baselines that isolate what actually matters in your system?

Novelty & Taste

Did you bring an idea we didn't expect? Surprise us.

 **Self-evaluation is required.** Define how you evaluate your own system—what metrics, what criteria, what test cases reveal about quality. Show us you can distinguish between a system that generates plausible output and one that generates correct output. If you can't tell us where your system is weak, we'll assume you don't know.

What else matters beyond these? That's worth thinking about too.

5. Submission

 Please share your private repo at least one day ahead of your presentation day with shimingren@reinforcelabs.ai so we have time to review your code, repo structure, and prepare questions.

Private GitHub repo shared with shimingren@reinforcelabs.ai —code, README, and example outputs

Presentation materials (slides, notebook, or your preferred format)

Design document covering your full architecture, key decisions, and assumptions

We will ask about design choices during the interview.

- ❗ Questions about timeline, format, or submission? Happy to help. Questions about how to design your system? That's on you—making those calls is the exercise.

Reinforce Labs · Research Engineer Take-Home Assessment